Puntatori

Salvatore Filippone
salvatore.filippone@uniroma2.it

Federica Montesano
federica.montesano@uniroma2.it



Indice dei Contenuti

1 Introduzione ai Puntatori

2 Differenza con gli Array

3 Puntatori e Matrici

4 Memoria Dinamica

5 Utilizzo dei Puntatori

Roadmap

1 Introduzione ai Puntatori

2 Differenza con gli Array

3 Puntatori e Matrici

4 Memoria Dinamica

5 Utilizzo dei Puntatori

Cos'è un Puntatore?

Definizione

Un **puntatore** in C è una variabile che memorizza come valore l'**indirizzo di memoria** di un'altra variabile o risorsa all'interno dello spazio di indirizzamento del processo.

Operatori Fondamentali:

- `&` (*Address-of*): Restituisce l'indirizzo fisico/logico di memoria di una variabile.
- `*` (*Dereference*): Accede al valore memorizzato all'indirizzo contenuto nel puntatore.

Rappresentazione concettuale

```
int x = 20;
int *p = &x;
```

Variabile	Indirizzo	Valore
x	0x7fff10	20
p	0x7fff18	0x7fff10

Sintassi, Uso e Tipologie di Puntatori

Sintassi ed Esempio Pratico

```
1  int x = 10;
2  // Puntatore a un intero
3  int *ptr = &x;  // Indirizzo (&)
4  int val = *ptr; // Dereferenziazione (*)
5
6  // Puntatore Generico (void*)
7  void *gptr = (void *)&x;
8  val = (int)*gptr; // Cast esplicito
9
10 // Aritmetica dei puntatori
11 ptr++;          // Avanza di sizeof(int)
12 val = *(ptr+1); // Equivalente a ptr[1]
```

Tipologie Speciali di Puntatori

- **NULL Pointer**: Inizializzato a `NULL`; indica l'assenza di un indirizzo valido.
- **Wild Pointer**: Puntatore non inizializzato contenente un valore casuale ("spazzatura").
- **Dangling Pointer**: Puntatore a una zona di memoria gia deallocata o non piu valida.

Principi dell'Aritmetica dei Puntatori

Formula dell'Aritmetica degli Indirizzi

Se un puntatore `ptr` contiene un indirizzo ADDR e punta al tipo $type$:

$$\text{Nuovo Indirizzo} = \text{ADDR} \pm (n \times \text{sizeof}(type))$$

Operazioni Consentite

- **Inc/Dec:** `ptr++`, `ptr-`
- **Offset:** `ptr + n`, `ptr - n`
- **Differenza:** `p2 - p1` ($\rightarrow$ `ptrdiff_t`)
- **Confronto:** `p1 == p2`, `p1 < p2`

Operazioni VIETATE

- **Somma tra puntatori:** `p1 + p2`
- **Moltiplicazione e Divisione**
- **Operazioni Bitwise:** `&`, `|`, `~`
- Assegnazione tra tipi incompatibili

Esempio Pratico: Aritmetica dei Puntatori

```c
int arr[3] = {10, 20, 30};
int *p1 = arr;      // Indirizzo di arr[0]
int *p2 = &arr[2]; // Indirizzo di arr[2]

// 1. Offset (avanza di sizeof(int) byte)
printf("p1 + 1: %p\n", (void*)(p1 + 1));

// 2. Differenza (tipo ptrdiff_t)
ptrdiff_t diff = p2 - p1;
printf("Distanza: %td elementi\n", diff); // 2

// 3. Confronto relazionale
if (p1 < p2) {
    printf("p1 precede p2 in memoria\n");
}
```

Note di Esecuzione

- **Offset:** `p1 + 1` incrementa l'indirizzo di $1 \times$ sizeof(int) (es. +4 byte).
- **Differenza:** Ritorna la distanza in **numero di elementi**, non in byte.
- **Confronto:** Utile per verificare i limiti di un array nei cicli.

Roadmap

1 Introduzione ai Puntatori

2 Differenza con gli Array

3 Puntatori e Matrici

4 Memoria Dinamica

5 Utilizzo dei Puntatori

Array vs Puntatori

In C l'indicizzazione di un array (`arr[i]`) è uno *zucchero sintattico* dell'aritmetica dei puntatori (`*(arr + i)`).

Equivalenza del Compilatore

Per qualsiasi array o puntatore, l'accesso tramite indici viene tradotto come:

$$\texttt{arr[i]} \iff \texttt{*(arr + i)}$$

- Il nome dell'array `arr` viene valutato come l'indirizzo del suo primo elemento `&arr[0]`.
- Poiché la somma gode della proprietà commutativa, l'espressione `*(i + arr)` è ugualmente valida $\implies$ la sintassi `i[arr]` è equivalente ad `arr[i]`!

Relazione tra Array e Puntatori

In C, il nome di un array si riferisce (quasi) sempre al puntatore al suo primo elemento ($\&a[0]$).

```c
int a[5] = {10, 20, 30, 40, 50};
int *p = a; // Equivalente a: int *p = &a[0];

// Le seguenti espressioni accedono tutte allo STESSO elemento (es. il terzo):
int val1 = a[2];       // Notazione ad indici standard
int val2 = *(a + 2);  // Aritmetica dei puntatori applicata all'array
```

Array vs Puntatori

Caratteristica	**Array (`type arr[N]`)**	**Puntatore (`type *ptr`)**
Natura	Blocco contiguo e fisso di memoria.	Variabile contenente un indirizzo.
Dimensione	`sizeof(arr)` = $N \times$ `sizeof(T)`	`sizeof(ptr)` = 4 o 8 byte (su 64-bit).
Riassegnazione	**Impossibile** (`arr` è un indirizzo costante).	**Possibile** (`ptr = altro_ptr;`).
Allocazione	Automatica (Stack) o Statica.	Può puntare a Stack o Heap (`malloc`).

Array Decay (Decadimento dell'Array)

Cos'è l'Array Decay?

Quando un array viene passato a una funzione, converte implicitamente il suo tipo in un **puntatore al primo elemento** (`type arr[]` $\rightarrow$ `type *`), perdendo l'informazione sulla dimensione totale.

```
1  void f(int arr[]) {
2      // arr[] è int*: cosa restituisce?
3      printf("%lu\n", sizeof(arr));
4  }
5
6  int main() {
7      int v[10]; // sizeof(v) == 40 byte
8      f(v);      // v decade a int*
9  }
```

Eccezioni

L'array **non** decade solo se usato con:

- `sizeof(arr)`: dimensione totale in byte.
- `&arr`: indirizzo dell'intero array.

Roadmap

1 Introduzione ai Puntatori

2 Differenza con gli Array

3 Puntatori e Matrici

4 Memoria Dinamica

5 Utilizzo dei Puntatori

Puntatore a Puntatore (Doppio Puntatore)

```c
int val = 100;
int *ptr = &val;   // Puntatore a int
int **pptr = &ptr; // Puntatore a puntatore

// Accesso al valore (100)
printf("val: %d | *ptr: %d | pptr: %d\n",
val, *ptr, **pptr);

// Relazione tra indirizzi
printf("&val:  %p\n", (void*)&val);
printf("ptr:   %p (uguale a &val)\n", (void*)ptr);
printf("&ptr:  %p\n", (void*)&ptr);
printf("pptr:  %p (uguale a &ptr)\n", (void*)pptr);
```

Concetto e Catena

Un doppio puntatore (`type **`) memorizza l'indirizzo di memoria di un altro puntatore:

$$\texttt{pptr} \rightarrow \texttt{ptr} \rightarrow \texttt{val}$$

Casi d'Uso Principali

- **Modifica di puntatori** in funzioni (passaggio per riferimento).
- **Allocazione dinamica 2D** (array di puntatori o matrici dinamiche).

Matrici 2D e Notazione Puntatori

Una matrice 2D `type mat[M][N]` è memorizzata in modo contiguo in memoria secondo il **Row-Major Order**.

Accesso agli Elementi di una Matrice 2D

L'elemento alla riga i e colonna j può essere espresso sia con la notazione ad indici sia con la dereferenziazione multipla dei puntatori:

$$\texttt{mat[i][j]} \iff \texttt{*(*(mat + i) + j)}$$

- `mat + i`: Avanza alla riga i-esima (incrementando l'indirizzo di $i \times N \times$ `sizeof(type)`).
- `*(mat + i)`: Ottiene l'indirizzo del primo elemento della riga i.
- `*(mat + i) + j`: Ottiene l'indirizzo dell'elemento (i, j).

Roadmap

1 Introduzione ai Puntatori

2 Differenza con gli Array

3 Puntatori e Matrici

4 Memoria Dinamica

5 Utilizzo dei Puntatori

Puntatori e Memoria Dinamica

Come gestire necessità di memoria di dimensioni non prevedibili al momento della scrittura del programma? Ovvero, come si può ottenere memoria durante l'esecuzione di un programma?

- Ottenere memoria con `malloc`:

```
void *malloc(size_t size);
```

- Rilasciare la memoria con

```
void free(void *ptr);
```

Esempio: se abbiamo bisogno di un'area di memoria composta da `K` numeri interi, possiamo eseguire:

```
int  *a;
a = (int *) malloc(K*sizeof(int));
```

Layout di Memoria di un Programma C

I 5 Segmenti della Memoria

1. **Text (Code):** Codice eseguibile (solo lettura).
2. **Initialized Data:** Variabili globali/statiche inizializzate.
3. **BSS (Uninitialized Data):** Variabili globali/statiche non inizializzate (azzerate a runtime).
4. **Heap:** Memoria dinamica (`malloc`), cresce verso l'alto.
5. **Stack:** Variabili locali e frame di funzione, cresce verso il basso.

Mappa della Memoria (RAM)



Dove Risiedono le Variabili?

```c
#include <stdio.h>
#include <stdlib.h>

int g_init = 10;     // Data Segment
int g_uninit;        // BSS Segment

void func() {
    static int s = 5; // Data Segment
    int local = 10;   // Stack Segment
}

int main() {
    int *p = malloc(sizeof(int)); // Heap
    free(p);
    return 0;
}
```

Riepilogo Allocazioni

- **Globali/Statiche Inizializzate:** Data Segment.
- **Globali/Statiche NON Inizializzate:** BSS Segment.
- **Variabili Locali & Argomenti:** Stack Segment.
- **Memoria Dinamica:** Heap Segment (tramite puntatore sullo Stack).

Gestione della Memoria: Stack vs Heap

Memoria Stack

Cos'è: Memoria automatica e gestita direttamente dal compilatore. È più veloce ma di dimensione limitata. Le variabili vengono distrutte appena si esce dalla funzione.

Esempio Stack

```
void esempioStack() {
int x = 10;     // Allocata nello Stack
int v[100];     // Dimensione fissa
} // x e v vengono eliminate
// automaticamente qui!
```

Memoria Heap

Cos'è: Memoria dinamica di dimensioni medio-grandi, gestita dal programmatore. Permette di decidere la dimensione durante l'esecuzione e persiste finché non la si libera esplicitamente.

Esempio Heap

```
void esempioHeap(int n) {
// Alloca n interi nello Heap
int *p = malloc(n * sizeof(int));

// Deve essere liberata a mano!
free(p);

}
```

Funzioni di Libreria per la Memoria Dinamica

malloc vs calloc

- `malloc(size)`: Alloca la memoria senza inizializzarla.
- `calloc(n, size)`: Alloca memoria per n elementi e la inizializza con zeri.

```
// Valori non inizializzati
int *p1 = malloc(5 * sizeof(int));

// Memoria azzerata (tutti 0)
int *p2 = calloc(5, sizeof(int));
```

realloc e free

- `realloc(ptr, new_size)`: Ridimensiona un blocco esistente mantenendo i dati precedenti.
- `free(ptr)`: Rilascia la memoria nell'Heap rendendola di nuovo disponibile.

```
// Ridimensiona p1 a 10 interi
int *temp = realloc(p1, 10 * sizeof(int));
if (temp != NULL) p1 = temp;

// Rilascio della memoria
free(p1); free(p2);
```

Errori Comuni

1. Mancato Controllo NULL

In alcuni casi, `malloc` fallisce restituendo `NULL`. Dereferenziare un puntatore 'NULL' causa **Segmentation Fault**.

```
int *p = malloc(K * sizeof(int));
if (p == NULL) {
    fprintf(stderr, "Errore: OOM!\n");
    exit(1);
}
```

2. Double Free

Chiamare `free()` due volte sullo stesso indirizzo causa crash o vulnerabilità di sicurezza.

```
int *a = malloc(sizeof(int));
int *b = a;
free(a);
free(b); // ERRORE: Double Free!
```

3. Memory Leak

Memoria allocata e non liberata prima di perdere il puntatore che la riferisce.

```
for (int i = 0; i < 10; i++) {
    // Il vecchio indirizzo viene sovrascritto:
    // la memoria precedente e' persa per sempre!
    p = malloc(K * sizeof(int));
}
```

Gestione delle Stringhe in Memoria

Punti Critici

- **Terminatore `'\0'`:** Allocare sempre 1 byte in più oltre alla lunghezza della stringa (`strlen(s) + 1`).
- **Copia Sicura:** Usare funzioni come `strncpy` o `snprintf` per prevenire buffer overflow.
- **Manipolazione:** Concatenazione (`strcat`) e tokenizzazione (`strtok`) modificano direttamente il buffer di memoria.

Esempio Allocazione Stringa

```c
char *src = "Hello";

// +1 per il carattere '\0'
char *dst = malloc(strlen(src) + 1);

if (dst != NULL) {
    strcpy(dst, src);
    // ...
    free(dst);
}
```

Roadmap

1 Introduzione ai Puntatori

2 Differenza con gli Array

3 Puntatori e Matrici

4 Memoria Dinamica

5 Utilizzo dei Puntatori

Passaggio Parametri: Valore vs Riferimento

Passaggio per Valore (Copia)

In C i parametri sono passati per valore: la funzione lavora solo su una copia.

```c
void incValore(int a) {
    a = a + 1; // Modifica locale
}

int main() {
    int n = 10;
    incValore(n);
    printf("%d\n", n); // Output: 10
}
```

Passaggio per Riferimento

Passando l'indirizzo (`&n`), la funzione agisce direttamente sulla variabile originale.

```c
void incRiferimento(int *a) {
    *a = *a + 1; // Modifica originale
}

int main() {
    int n = 10;
    incRiferimento(&n);
    printf("%d\n", n); // Output: 11
}
```

Modificare un Puntatore in una Funzione

Perché serve un doppio puntatore (type **)?

Per allocare memoria o modificare un puntatore all'interno di una funzione, è necessario passarne l'indirizzo di memoria tramite un doppio puntatore.

ERRATO: Passaggio di int*

```
void allocaErrato(int *v) {
    // Modifica solo la copia locale!
    v = malloc(10 * sizeof(int));
}
// Memory leak: il chiamante perde
// il riferimento all'allocazione
```

CORRETTO: Passaggio di int**

```
void allocaCorretto(int **v) {
    // Modifica il puntatore originale
    *v = malloc(10 * sizeof(int));
}

int main() {
    int *v = NULL;
    allocaCorretto(&v); // Passa &v
    free(v);
}
```